Guía para uso de computadoras ...

¿Cómo saber si compila o no?

- mvnw.cmd compile

Si el resultado final es **BUILD SUCCESS**, tu código compila perfectamente. Si pone **BUILD FAILURE**, revisa los errores que muestra la terminal.

¿Probar un test independiente?

O aquí  o pones en la terminal ./mvnw test -Dtest=Test7Alt

¿Como hacer commit dsp de cada ejercicio?

git status
git add .
git commit -m "Completado ejercicio X"

¿Como hacer push dsp de acabar el examen?

git push (entrar desde el navegador y comprobar que se han hecho todos los cambios)

Anotaciones:

Anotaciones de la clase

- Anotar con @Getter y @Setter
- Anotar siempre con @Entity
- Anotar con @Table(name = “xxxxx”)(acabarlo en s para luego que no dé error en los insert)

Anotaciones de los atributos

- @NotNull comprueba que un atributo no es null. Es decir, comprueba que un atributo no tiene ningún valor.
- @NotEmpty comprueba que un atributo, ya sea un String o una lista no está vacío.
- @NotBlank comprueba que un atributo no es null y tampoco está vacío.
- @Size(min = x, max = y) nos permite especificar la longitud máxima y mínima para

un String
-@DecimalMin (decimales coma flotante)
- @Min(x) y @Max(y) nos permite especificar el valor máximo y mínimo que puede tomar un valor numérico
- Pattern(reg_exp = “xxxx”)
-@Column(name = “nombre”)(unique = true): por si lo quieres hacer único
- @ManyToMany(cascade = CascadeType.xxx)
Set<ClaseRelacionada> nombreClase
- @Enumerated(EnumType.STRING) para los enums
- Si es opcional, basta con no ponerle @NotNull
-@Positive o @PositiveOrZero
-@Email: para emails

Relaciones

Anotación	Significado	Ejemplo en tu proyecto
`@ManyToOne`	Muchos de A apuntan a un B	`ChessMatch.white`, y `challenge` del Test 2
`@OneToMany`	Un A tiene varios B	`Owner.pets` en las diapositivas (con `mappedBy`)
`@OneToOne`	Un A tiene un B y viceversa	`ChessMatch.board`
`@ManyToMany`	Muchos A con muchos B	`FriendlyChallenge.participants`, y `recipients` del Test 2

La regla para elegir es mirar la cardinalidad del UML en el extremo al que apunta la flecha:

- Si ese extremo es `1` o `0..1`, es `@ManyToOne` (o `@OneToOne` si el otro lado también es uno solo).
- Si es `0..n` o `2..n`, es `@OneToMany` o `@ManyToMany`.

Parámetros útiles

- `optional = false` (en `@ManyToOne` y `@OneToOne`): la relación es obligatoria. Por defecto es `true`.
- `mappedBy = "atributo"` (en `@OneToMany` y el lado inverso de las bidireccionales): indica qué atributo del otro lado es el dueño de la relación. En el Test 2 no lo necesitas, porque todas son unidireccionales.
- `cascade = CascadeType.ALL`: las operaciones (guardar, borrar) se propagan a los objetos relacionados. Se usa en composiciones, como `ChessMatch.board`. T4 muestra `ALL` con `orphanRemoval = true` para composiciones, y `PERSIST`, `MERGE`, `DETACH` y `REFRESH` para agregaciones.
- `orphanRemoval = true`: borra los hijos que dejan de estar en la colección.
- `fetch = FetchType.LAZY` o `EAGER`: cuándo se carga el objeto relacionado.

- si te dicen que la cardinalidad es 1:
  (optional = false)

Insercciones:

- Fijarse en el modelado conceptual
- Poner el nombre de las tablas en el ej1 en plural para luego hacer los insert en camel_case / snake_case
- Poner el enum / Mensajes en inglés (mirarlo en el modelado)
- Siempre comillas simples, si se usa can't por ejemplo se pone: can''t (2 comillas simples)
- El nombre de los atributos es, o el mismo de su clase o el mismo + _id
- Recordar " , " al final de cada insert y " ; " dsp del último insert

- Mirar en el debug console los nombre de las tablas, atributos, valores de los enums (todo viene hecho ahí)

Tablas intermedias: (relaciones ManyToMany)

Nombre: <tabla_dueña>_<campo> (no se incluye los <>)

- Los atributos seguramente sean tabla_dueña(sin la s)_id y campo_id

Servicios:

- Poner @Service arriba
- Poner @autowired encima del repo
- Poner extends CrudRepository en el repo asociado
- Usar @Transactional(readOnly = true), (si no funciona, está importado el transactional que no es -importar desde org.springframework…) para (`findAll`, `findById`, `findByUsername`, consultas con `@Query` de tipo `SELECT`)
- (rollbackFor = InvalidWinnerException.class), por si se necesita lanzar una excepción (no tiene que ser siempre ese tipo/clase, puede ser otra)
- Incluir en la cabecera de la función: `throws UnfeasibleAttemptException`
- Usar mejor .equals(object o) antes que == (principio de igualdad / identidad)
- Basarse en otras clases de service para ver como se hace, y ver la clase principal para saber los métodos que tiene (getters y setters)
- Que el delete no se llame a él mismo sino al del repositorio:
  java.lang.StackOverflowError
- Poner !(m.getFinish().isAfter(m.getStart())) = tema de >=

```java
@Transactional(rollbackFor = UnfeasibleAttemptException.class)
public ChessProblemAttempt save(ChessProblemAttempt m) throws UnfeasibleAttemptException {
    //El instante de inicio debe ser anterior al instante de fin en caso de que este no sea nulo.
    if (m.getStart() != null && m.getFinish() != null && m.getStart().isAfter(m.getFinish())) {
        throw new UnfeasibleAttemptException(message: "The start time must be before the end time.");
    }
    List<ChessProblemAttempt> activeAttempts = repo.findOngoingAttemptsByUserAndProblem(m.getPlayer(), m.getProblem());
    if (!activeAttempts.isEmpty() && !activeAttempts.get(index: 0).getId().equals(m.getId())) {
        throw new UnfeasibleAttemptException(message: "There is already an active attempt for this problem by the same player.");
    } else {
        return repo.save(m);
    }
```

```java
@Transactional(rollbackFor = ResourceNotFoundException.class)
public List<RankingEntry> getRanking(Integer courseId) {
    List<RankingEntry> ranking = new ArrayList<>();
    List<Object[]> results = repository.findStudentPoints(courseId);
    if (results == null || results.isEmpty()) {
        throw new ResourceNotFoundException("No ranking found for course with id: " + courseId);
    }
    else{
        for (Object[] result : results) {
            SchoolMember member = (SchoolMember) result[0];
            Long points = (Long) result[1];
            ranking.add(new RankingEntry(member, points));
        }
    }
    return ranking;
}
```

Controladores:

- Poner arriba @RestController
- Leer bien, hay veces que piden cambiar el service y el controlador (basarse en las funciones del service para hacer el controller)
- Importar clases de las excepciones
- Ver atributos de la clase que estamos tocando (no siempre funciones)
- No usar .get() -> Devuelve un error 500. Usar un:

```
 .orElseThrow(()-> new ResourceNotFoundException("Match not found"));
```

- Errores 404 (NOT_FOUND):

```
  .orElseThrow(() -> new ResourceNotFoundException("FriendlyChallenge", "id", challengeId));
```

Se le pasan 3 cosas al ResourceNotFoundException:

- resourceName: ¿Qué cosa no encontré?
- fieldName: ¿Por qué campo la buscaba?
- fieldValue: ¿Con qué valor en ese campo?

- Errores 403 (PROHIBIDO):

-AccessDeniedException (“Mensaje que quieras poner”)

-Que tenga la authority “PLAYER”:

Poner @RequestMapping(“ruta general”)

Poner @GetMapping (1:vacío, 2:”/{id}”)

Ir al SecurityConfiguration y añadir un

```
.requestMatchers(“ruta general/**”)
```

Estructura general (Se ha puesto arriba un RequestMapping(“ruta general”))

```java
//devolver todos los intentos existentes
@GetMapping ()
public ResponseEntity<List<ChessProblemAttempt>> getAllAttempts() {
    return new ResponseEntity<>(matchService.getAttempts(), HttpStatus.OK);
}

@GetMapping ("/{id}")
public ChessProblemAttempt getAttemptById(@PathVariable ("id")Integer id) {
    return matchService.getMatchById(id).orElseThrow(() -> new ResourceNotFoundException(message: "Attempt not found"));
}
```

- Cuando te pida devolver en un formato JSON, crear clase DTO de respuesta:
    - Crear clase con el nombre, poner @Getter y @Setter
    - Poner todos los atributos en privado
    - Crear constructor: public static nombre clase from(Si necesito otras clases)
    - Creo una instancia de la clase y la lleno con las otras clases y devuelvo el dto

Creación componente básico (FrontEnd):

- cd frontend, npm install y para ejecutar npm test y luego pulsar la a en el menú
- Mirar en los test para sacar el nombre de las propiedades y que te pide cada uno
- Importar useEffect y useState
- Declarar const
- Usar useState, que varíe si cambia el parámetro de la función
- Usar fetch para una URL
    - Añadir dsp del fetch esto:

        .then((res) => res.json())

        .then((data) => setRanking(data));
- Añadir en el return lo que quieras printear

- Estructura para usar .map:
    - `.map` recorre un array y **crea un array nuevo** con el resultado de aplicar una función a cada elemento. El array original no se modifica, y el nuevo tiene siempre **la misma longitud**.

```
>
 {ranking.map((e) => (
     <li key={e.user.id} id={`ranking-${e.user.id}`}>
         {`${e.user.username} - ${e.points}`}
     </li>
 ))}
l>
```

- Se usa li (elemento de la lista) pq lo dice el enunciado
- Ver el test para ver a donde puede acceder cada elemento del array (en nuestro caso e)
- El key se tiene que poner siempre, pasan los test sin él, pero te saldrá un warning. La key debe ser algo que identifique a cada elemento de forma única dentro de esa lista, y lo ideal es que salga de los propios datos (normalmente un id)
- El id = , es pq el test lo exige con estas lineas:

```
items.forEach((item) => {
  expect(item).toHaveAttribute('id');
});
```

En este caso, el valor no importa mucho cual sea

Sería lo mismo poner:

id={entry.user.id}

id={`ranking-${entry.user.id}`}

id={entry.user.username}

Sería distinto si en el test pusiera esto:

expect(item).toHaveAttribute('id', 'ranking-4');

Estructura general (tablas) (en la siguiente página):

```jsx
import React, { useEffect, useState } from "react";
import { Table } from "reactstrap";

export default function AttemptsListing() {
    const [matches, setMatches] = useState([]);

    useEffect(() => {
        // Simulate an API call to fetch attempts
        const fetchAttempts = async () => {
            // Replace this with your actual API call
            const response = await fetch('/api/v1/attempts');
            const data = await response.json();
            setMatches(data);
        };

        fetchAttempts();
    }, []);

    if(matches.length === 0) {
        return <h1>NO ATTEMPTS</h1>;
    }

    return (
        <Table>
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Start</th>
                    <th>Finish</th>
                    <th>Player</th>
                    <th>Problem</th>
                    <th>Difficulty</th>
                </tr>
            </thead>
            <tbody>
                {matches.map((match) => (
                    <tr key={match.id}>
                        <td>{match.name}</td>
                        <td>{match.start}</td>
                        <td>{match.finish}</td>
                        <td>{match.player.username}</td>
                        <td>{match.problem.name}</td>
                        <td>{match.problem.difficultyLevel}</td>
                    </tr>
                ))}
            </tbody>
        </Table>
    );
}
```

```java
@Transactional(readOnly = true)
public List<RankingEntry> getRanking(Integer courseId) {
    if (!chessCourseRepository.existsById(courseId)) {
        throw new ResourceNotFoundException("ChessCourse", "id", courseId);
    }

    List<Object[]> rawRanking = chessCourseRepository.findStudentPoints(courseId
    List<RankingEntry> ranking = new ArrayList<>();

    if (rawRanking != null) {
        for (Object[] row : rawRanking) {
            SchoolMember member = (SchoolMember) row[0];
            Number pointsNumber = (Number) row[1];
            Integer points = pointsNumber != null ? pointsNumber.intValue() : 0;
            ranking.add(new RankingEntry(member, points));
        }
    }

    // Ordenar de mayor a menor puntuación
    ranking.sort((entry1, entry2) -> Integer.compare(entry2.getPoints(), entry1.

    return ranking;
}
```

```jsx
export default function RankingTable({ ranking }) {
  return (
    <Table>
      <thead>
        <tr>
          <th>Position</th>
          <th>Student</th>
          <th>Points</th>
        </tr>
      </thead>
      <tbody>
        {ranking.map((entry, index) => (
          <tr key={entry.member.id}>
            <td>{index + 1}</td>
            <td>{entry.member.username}</td>
            <td>{entry.points}</td>
          </tr>
        ))}
      </tbody>
    </Table>
  );
}
```



```jsx
export default function CourseRanking({ courseId }) {
  const [ranking, setRanking] = useState([]);

  useEffect(() => {
    const fetchRanking = async () => {
      try {
        const data = await getCourseRanking(courseId);
        setRanking(data);
      } catch (error) {
        setRanking([]);
      }
    };

    fetchRanking();
  }, [courseId]);

  if (!ranking || ranking.length === 0) {
    return <h1>NO RANKING</h1>;
  }

  return <RankingTable ranking={ranking} />;
}
```